ARCHITECTURE ANALYSIS & REBUILD OPTIONS

NZ Inventory Regional Health

A senior systems engineer's review of the current production app, five candidate rebuild architectures on AWS, Azure, and GCP, and two rebuild prompts ready to hand to a fresh development team.

Prepared by
Senior software designer & systems engineer

Date
20 April 2026

Current stack
Next.js 15 · Prisma 5 · Postgres · AWS Fargate

Scope
Full codebase audit, cloud options, rebuild prompts

Target clouds
AWS · Azure · GCP

Report length
9 sections · diagrams · comparison matrix

Contents

0 Executive Summary

1 Complete Feature & Functionality Inventory

2 Current Architecture — Honest Assessment

3 The Five Candidate Architectures

4 Side-by-Side Comparison

5 Final Recommendation — Top Two

6 Rebuild Prompt — Option A (AWS Refined)

7 Rebuild Prompt — Option B (GCP Cloud Run)

8 Appendix — Caveats

9 Appendix — Short-Term Fixes If Not Rebuilding Yet

0. Executive Summary

The app is a **multi-tenant B2B distributor ERP** for a New Zealand health-products company. It is approximately **45 pages**, **118 server actions** across 15 action files, **15 distinct reports** (CSV/XLSX/PDF), **30+ Prisma models**, **4 daily cron jobs** pinned to NZT, and a **QuickBooks Online** integration with AES-256-GCM-encrypted refresh tokens. Traffic is small (low tens of users), but report generation is CPU-heavy and transactional writes (inventory moves, reservations, landed-cost allocation) must be correct.

The current deployment works but has three material problems:

Three production gaps in the current stack

1. **ALB is HTTP only**, not HTTPS. Cookies, passwords, and QBO OAuth tokens traverse the public internet unencrypted.
2. **Cron jobs are declared in `vercel.json`** but the app runs on AWS ECS. Daily FX refresh, low-stock alerts, QBO sync, and scheduled reports never fire.
3. **Secrets live in plain ECS task-definition env vars.** `SESSION_SECRET`, `QBO_ENCRYPTION_KEY`, `DATABASE_URL` are readable by anyone with `ecs:DescribeTaskDefinition`.

Before recommending a rebuild, I note these three issues can all be fixed in a one-week incremental pass on the current stack. A rebuild is only justified if we also want **lower TCO**, **better cold-start economics**, **cleaner separation of concerns**, or **a different cloud for procurement reasons**.

Final recommendation

A **AWS Refined** GOLD PICK

ECS Fargate + HTTPS ALB + RDS + EventBridge Scheduler + Secrets Manager + S3. Lowest migration cost, retains existing SST/Pulumi IaC, closes all three production gaps, preserves Next.js server actions.

~$95/mo | ~5 days to ship | Zero code rewrite

B **GCP Cloud Run** SILVER PICK

Cloud Run + Cloud SQL + Cloud Scheduler + Secret Manager + Cloud Storage. Scale-to-zero economics, simplest operator model, managed HTTPS out of the box.

~$55/mo | ~5–7 days to ship | Cold start <1 s

The two rebuild prompts in sections 6 and 7 are self-contained and ready to hand to a development team.

1. Complete Feature & Functionality Inventory

Every capability here maps to real files in the repo — `app/`, `actions/`, `lib/`, `prisma/schema.prisma`, `app/api/`. No marketing text, no guesses.

1.1 Identity, Access, and Multi-Tenancy

AUTH

iron-session v8 cookie, bcryptjs rounds=10. Seal verified server-side in `lib/auth.ts`. Cookie `nz_inv_session`, HttpOnly, SameSite=Lax, 30-day TTL.

LOGIN

`/login` email + password → `signInAction`. No reset email, no rate limiting (gap).

BOOTSTRAP

`prisma/set-owner-password.ts` — idempotent ADMIN user create/rotate.

MIDDLEWARE

`middleware.ts` cookie-presence redirect; public paths for `/login`, QBO callback, cron, health.

ROLES

ADMIN, MANAGER, STAFF, VIEWER. `requireRole()` helper.

MULTI-TENANCY

Every row has `tenantId`. Enforced in app code only — no Postgres RLS.

1.2 Catalog — Products, Pricing, Dimensions

PRODUCTS

CRUD at `/products`. SKU, name, cost, reorder point/qty, dimensions, weight, brand, channel, territory, image.

DIMENSIONS

Brand / Channel / Territory — three classification axes. `/settings/dimensions`.

PRICE GROUPS

Retail, Wholesale, Trade, VIP (seeded). `/settings/price-groups`.

PRODUCT PRICES

`ProductPrice` joins Product × PriceGroup. NZD + effective date.

STOCK

`StockLevel` per product: qtyOnHand, qtyReserved, qtyIncoming, derived qtyAvailable / qtyFree.

BATCHES

`Batch` for batch-tracked products. Used by expiry tracker.

1.3 Inventory Movement

- **Ledger:** `InventoryTransaction` rows for RECEIVE, SHIP, ADJUST, TRANSFER, RESERVE, RELEASE.
- **Adjustments:** `/inventory` page — product search, qty delta, reason. Atomic with StockLevel.
- **Per-product views:** Levels / Transactions / Batches / Pricing / Sales History tabs.

1.4 Purchasing

- **PO lifecycle:** `DRAFT → SUBMITTED → PARTIAL → RECEIVED → CLOSED → CANCELLED`.

- **Foreign currency:** PO header stores currency, fxRate, fxRateDate. Stock costed in NZD at receipt.
- **Landed cost:** `PoReceiveCharge` for freight/duty/insurance/other, allocated by value or volume in `lib/inventory.ts`.
- **Receive flow:** RECEIVE transactions + stock increments + status transition.
- **CSV export:** `/api/purchase-orders/[id]/csv` for supplier hand-off.

1.5 Sales

- **Customers:** name, GST number, billing/shipping addresses, payment terms, price group, credit limit, contacts.
- **SO lifecycle:** `DRAFT → CONFIRMED → PICKING → SHIPPED → INVOICED → CANCELLED`.
- **Reservations:** Confirm creates `StockReservation` rows, reducing available without touching on-hand.
- **Shipping:** `shipSalesOrder` — decrements on-hand, releases reservations, writes SHIP transactions.
- **Invoicing + PDFs:** `@react-pdf/renderer` at `/api/sales-orders/[id]/invoice`.
- **Exports:** XLSX per SO, packing slip PDFs via `lib/pdf.tsx`.

1.6 Proforma Invoices & Credit Notes

- **Proforma:** `/proforma` for quotes & pre-payment invoices. Convertible to SO.
- **Credit notes:** linked to an originating SO; optional inventory reversal; PDF rendering.

1.7 Suppliers

- Supplier master + contacts; country, lead-time days, default currency, bank details, tax number.
- Supplier performance: on-time %, fill rate, price stability (`lib/reports/supplier.ts`).

1.8 Dashboards

- Root `/`: KPI tiles (revenue MTD, profit MTD, open POs/SOs, low stock), recent activity, stocked-value by brand (Recharts).
- `/reports` overview: revenue / cost / margin trend for fiscal year.

1.9 Reports (15 distinct)

ACTUAL-VS-BUDGET
Monthly actual vs `Budget`, variance % red/green.

BRAND-BREAKDOWN
Revenue & margin by brand, stacked bar.

CHANNEL-TRENDS
Revenue by channel over fiscal months.

CONTAINER-PLANNING
Group PO lines by supplier shipment.

CUSTOMER-SALES
Per-customer revenue, gross margin, lines.

CUSTOMER-TRENDS
Top N customers, month-over-month.

EXPIRY-TRACKER

MONTHLY-SALES

OVERSTOCK

Batches expiring within X months.

Revenue / cost / margin per month.

Products above months-on-hand threshold.

REORDER-PLANNER
At/below reorder point, suggested qty.

REP-PERFORMANCE
Revenue & margin by sales rep.

STOCK-ON-HAND
Snapshot valued at latest cost.

STOCK-TURN
Annualised inventory turnover.

SUPPLIER-ETA
Open PO lines + expected arrivals.

TESTER-TRACKER
Tester products, consumption & remaining.

Compute in `lib/reports/` — `margin.ts`, `inventory.ts`, `supplier.ts`, `trends.ts` — shared helpers `xlsx.ts` (ExcelJS) and `pdf.tsx` (@react-pdf).

1.10 Settings (8 admin pages)

users | price-groups | dimensions | budgets | tax | quickbooks | reports | audit

1.11 QuickBooks Online Integration

- **OAuth 2.0:** `/api/qbo/connect` + `/api/qbo/callback`. Refresh token AES-256-GCM encrypted with `QBO_ENCRYPTION_KEY` (32-byte).
- **Sync queue:** `QboSyncJob` + `lib/quickbooks/sync.ts`. Pushes invoices, payments, customers. 5 retries with exponential backoff.
- **Daily drain:** `/api/cron/qbo-sync` at 08:00 NZT.

1.12 Cron Jobs (declared, currently not firing)

> Declared in `vercel.json` but the app runs on ECS. **These do not fire in production today.**

JOB	TIME (NZT)	PURPOSE
`/api/cron/qbo-sync`	08:00	Drain QBO sync queue
`/api/cron/low-stock`	19:00	Find products below reorder point, email via Resend
`/api/cron/fx-rates`	18:00	Fetch yesterday's ECB NZD rates from Frankfurter.app, upsert `ExchangeRate`
`/api/cron/reports`	07:00	Render due `ScheduledReport` rows, email, write `ReportDelivery`

1.13 Data Model Summary 30+ models, 717 lines in schema.prisma

TENANCY & IDENTITY

Tenant, User

CATALOG

Product, Brand, Channel, Territory, PriceGroup, ProductPrice

INVENTORY

StockLevel, InventoryTransaction, Batch, StockReservation

PURCHASING

Supplier, SupplierContact, PurchaseOrder, PurchaseOrderLine, PoReceiveCharge

SALES

Customer, CustomerContact, SalesOrder, SalesOrderLine

PROFORMA / CREDITS

ProformaInvoice + line, CreditNote + line

PLANNING & REPORTING

Budget, ReportSnapshot, ScheduledReport, ReportDelivery

EXTERNAL INTEGRATION

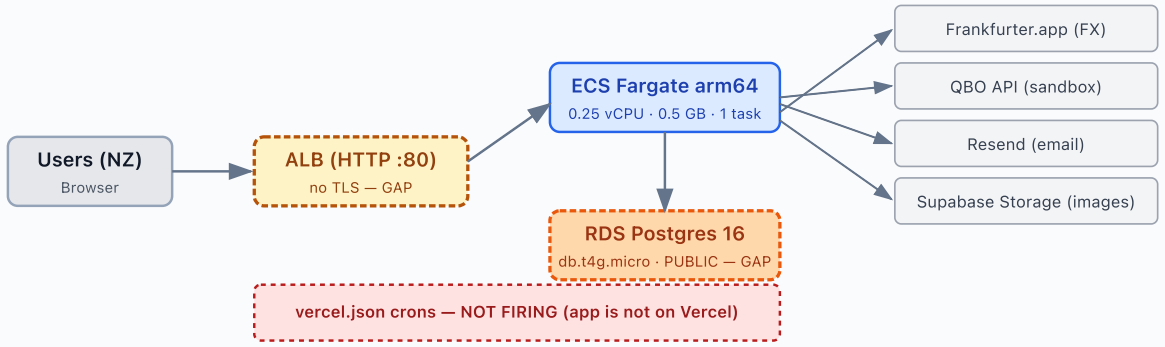
QboConnection, QboSyncJob, ExchangeRate

1.14 Non-Functional Behaviour

- **Fiscal year:** NZ standard April 1 → March 31.
- **Timezone:** NZT (Pacific/Auckland). All cron schedules and date formatting use `date-fns-tz`.
- **Currencies:** NZD base. POs in any currency, FX rate captured per PO.
- **Rounding:** GST & totals to 2 dp.
- **Testing:** Vitest in `__tests__/lib/` covers utils, currency, constants. Coverage is thin.

2. Current Architecture — Honest Assessment

Current topology



Current deployment in `ap-southeast-2`. Dashed red/amber boxes flag the production gaps.

What works

- Single codebase, single deployment unit. Next.js + Prisma on one container.
- Server Actions + Prisma combo is highly productive for CRUD-heavy ERP — 118 actions across 15 files.
- Multi-tenant enforcement consistent — every action starts with `requireSession()` and scopes by `tenantId`.
- Schema is clean — 30 models, proper `tenantId` indexes.
- IaC is TypeScript (SST/Pulumi). arm64 Fargate is ~20% cheaper than x86.

What doesn't work — the gap list

#	GAP	IMPACT	SEVERITY
1	ALB is HTTP only	Passwords & session cookies in the clear	Critical
2	Cron jobs don't fire (Vercel declarations on ECS)	No daily FX, no low-stock alerts, no QBO sync, no scheduled reports	Critical
3	Secrets in plain env	Anyone with `ecs:DescribeTaskDefinition` reads them	Critical
4	RDS publicly accessible	Only SG ingress rule protects Postgres port	High
5	`prisma db push --accept-data-loss` on every start	Bad schema edit = lost data on next deploy	High
6	No backups strategy beyond RDS defaults	Recovery is a prayer	High
7	No alarms / logging	No visibility into 5xx, DB saturation, cron failures	Medium
8	No CI/CD	Manual deploys = human error	Medium
9	Single-AZ RDS	Zone outage = downtime	Medium
10	Single Fargate task minimum	Crash loop = full outage	Medium

App profile — what matters for architecture choice

- **Low concurrent load** — single-digit users typically.
- **Bursty CPU** — report gen 1–5 s of @react-pdf / ExcelJS work.
- **Transactional writes** — stock moves, reservations, landed cost must be ACID.
- **Scheduled compute** — four fixed NZT crons daily.
- **Data-critical ledger** — the business lives in Postgres.
- **Single locality** — all users NZ/AU. No global CDN value.
- **Small team** — 1 dev. Ops overhead must be ≈ zero.

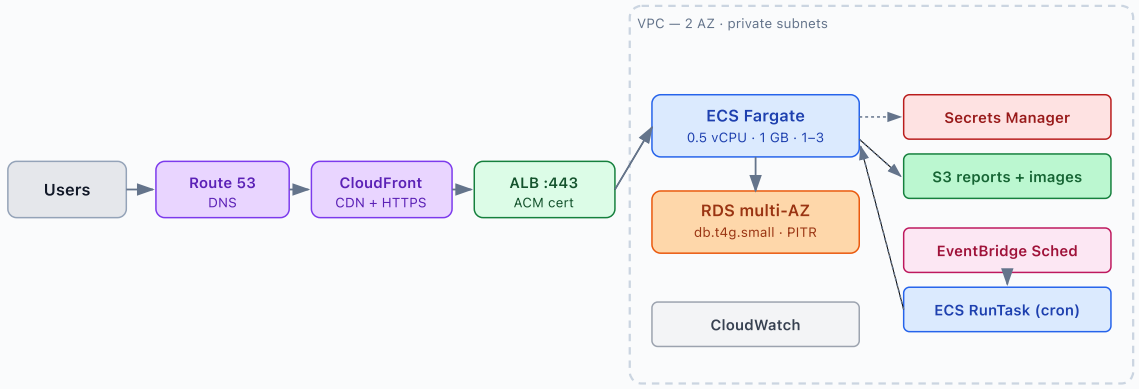
3. Five Candidate Architectures

Each option: topology, services, what changes, strengths, weaknesses, indicative monthly cost. Costs are on-demand list prices for `ap-southeast-2` / `australia-east` / `australia-southeast1` with no commitments.

1 AWS Refined — Container + RDS, done properly

RECOMMENDED · GOLD

Same shape as today, every rough edge removed.



Option 1 topology — existing shape with HTTPS, private DB, Secrets Manager, real cron, CloudWatch.

Strengths

- Smallest migration distance. Server Actions, Prisma, domain code unchanged.
- Retains team know-how; developer keeps AWS / SST / Pulumi.
- Closes every known production gap in one release.
- Multi-AZ RDS removes single-zone outage risk.
- arm64 Fargate keeps the price/performance advantage.

Weaknesses

- Always-on Fargate — pays for idle compute 24/7.
- Four-hop edge (Route 53 → CloudFront → ALB → Fargate) adds ~30 ms.
- NAT Gateway adds ~$32/mo once RDS is private.
- Broad operational surface area (10+ services in IaC).

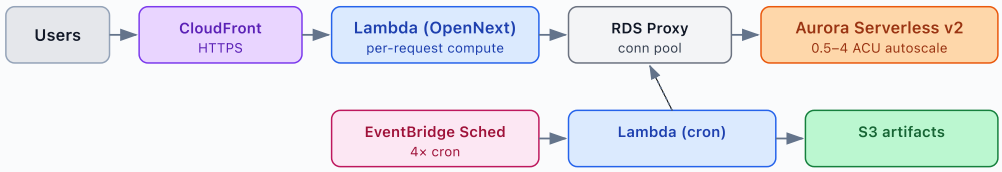
Indicative monthly cost

COMPONENT	$ / MONTH
Fargate 0.5 vCPU / 1 GB, 1 task	~18
ALB	~22

COMPONENT	$ / MONTH
CloudFront (low traffic)	~1
RDS db.t4g.small multi-AZ, 20 GB gp3	~45
NAT Gateway + data	~35
S3 + storage + requests	~2
Secrets Manager (6 secrets) + CW + Route 53 + ACM	~8
Total (multi-AZ)	**~$131**
Total (single-AZ — accept zone risk)	**~$95**

2 AWS Serverless — OpenNext + Aurora Serverless v2

Next.js runs on Lambda via OpenNext; Postgres is Aurora Serverless v2; no always-on compute.



Strengths

- True per-request compute billing; idle nights ≈ $0.
- Automatic scale-out for bursts.
- No container image to maintain.
- Aurora Serverless v2 scales to hundreds of tenants without re-architecting.

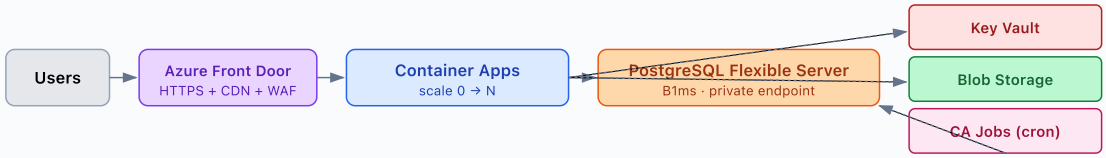
Weaknesses

- Cold starts: 1–3 s for first hit after idle.
- Aurora floor: 0.5 ACU min ≈ $43/mo (cannot pause).
- Prisma-on-Lambda quirks (connection limit, binary target).
- OpenNext is an extra abstraction that can break on Next.js minor versions.
- Observability fragmentation across Lambda log groups.

Indicative total: ~$70 / month (mostly Aurora + Proxy).

3 Azure Container Apps — Microsoft Stack

Container Apps (scale-to-zero) + Flexible Postgres + Container Apps Jobs for crons.



Strengths

- Scale-to-zero — idle overnight = storage only.
- Managed HTTPS via Front Door is a one-liner.
- CA Jobs is a cleaner cron primitive than EventBridge → RunTask.
- Enterprise fit: if NZ targets are M365-aligned, Entra ID SSO is easy.
- B1ms DB is cheap (~$18/mo).

Weaknesses

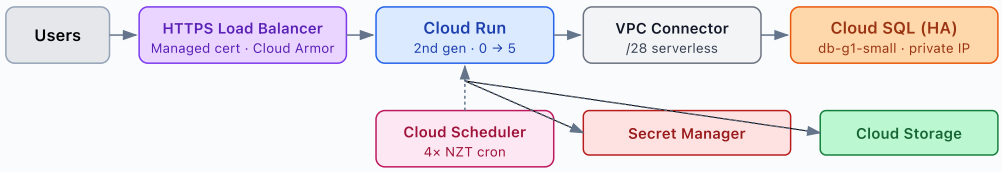
- Cold starts 1–3 s on scale-from-zero.
- Team has no stated Azure experience — ~1 week ramp.
- Front Door egress is higher than CloudFront at low volume.
- Private endpoint + VNet setup is more wizardry than AWS.
- Fewer reliable AI-agent MCPs, more portal clicks.

Indicative total: ~$40–74 / month depending on Front Door inclusion.

4 GCP Cloud Run + Cloud SQL — Simplest, Lowest TCO

RECOMMENDED · SILVER

Cloud Run containers + Cloud SQL + Cloud Scheduler pulling HTTPS endpoints.



Strengths

- Scale-to-zero with industry-leading cold starts (200–800 ms).
- Managed HTTPS is a one-liner; certs auto-issue.
- Cloud Scheduler is the cleanest cron primitive of the five.
- Lowest idle cost: ~$30/mo at genuinely low usage.
- Simplest operator model for a 1-dev team.

Weaknesses

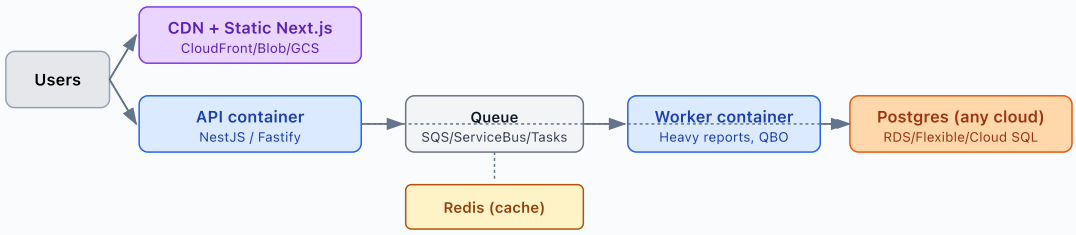
- Cloud SQL connector / VPC connector adds a step.
- Less NZ SME procurement familiarity than AWS/Azure.
- No NZ region — Sydney (australia-southeast1) is nearest.
- Team has no stated GCP experience — ~1 week ramp.

Indicative monthly cost

COMPONENT	$ / MONTH
Cloud Run (low CPU seconds)	~3
Cloud SQL db-g1-small HA, 20 GB SSD	~28
Serverless VPC Connector (min)	~10
HTTPS Load Balancer (global)	~18
Cloud Storage + Secret Manager + Logging	~4
Cloud Scheduler (4 jobs, free tier)	0
Egress	~2
Total (with Global HTTPS LB + HA)	**~$65**
Total (no LB, direct `run.app`)	**~$47**
Total (db-f1-micro, dev/single-tenant)	**~$28**

5 Split Architecture — Frontend + API + Queue Worker

Static frontend + dedicated API + background worker + queue. Cloud-agnostic.



Strengths

- Independent scaling — a slow report in the worker never blocks users.
- Cloud-portable — same images run on AWS, Azure, GCP.
- Clean REST surface is easier to test than scattered Server Actions.
- Long-running report jobs in worker, not HTTP handler.

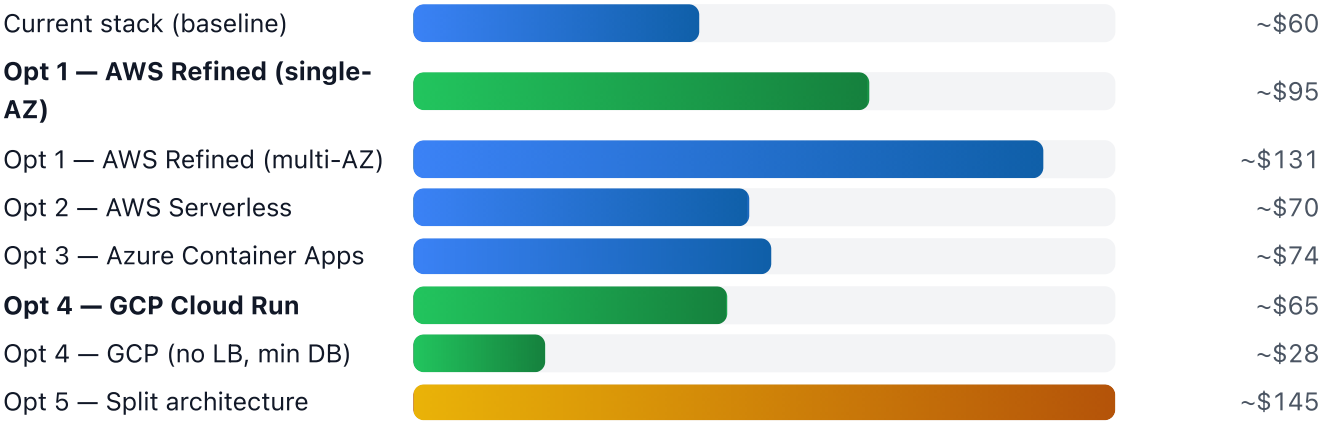
Weaknesses

- **Major rewrite — 8–12 weeks**. Every Server Action becomes a REST endpoint.
- Higher infra cost (~$145+/mo) — 3 containers + Redis + queue.
- Two codebases minimum; auth wired twice.
- Benefits only materialise at scale this app doesn't have.
- Abandons the productive Server Actions + Prisma model.

Indicative total: ~$145+ / month on AWS.

4. Side-by-Side Comparison

Monthly cost at current usage



Capability matrix

Legend: ✓✓ strong ✓ adequate ~ workable but painful ✕ poor fit

DIMENSION	OPT 1 AWS REFINED	OPT 2 AWS SERVERLESS	OPT 3 AZURE CA	OPT 4 GCP CLOUD RUN	OPT 5 SPLIT
Migration distance from current code	✓✓	~	~	~	✗
Preserves Server Actions	✓✓	✓	✓✓	✓✓	✗
Preserves Prisma schema & migrations	✓✓	✓	✓✓	✓✓	✓✓
Handles heavy PDF / XLSX gen	✓✓	~	✓✓	✓✓	✓✓
Cron support for 4 daily NZT jobs	✓	✓✓	✓✓	✓✓	✓✓
Scale to zero	✗	✓✓	✓✓	✓✓	~
Cold-start impact on UX	✓✓ none	~ 1-3s	~ 1-3s	✓ <1s	~
Managed HTTPS out of the box	✓	✓	✓✓	✓✓	✓
Secrets management	✓✓	✓✓	✓✓	✓✓	✓✓
Private database networking	✓✓	✓✓	✓✓	✓✓	✓✓
Multi-AZ / HA DB	✓✓	✓✓	✓	✓	✓✓
Observability	✓	~	✓	✓✓	✓✓
IaC reuse (SST/Pulumi already)	✓✓	✓	~	~	~
1-dev cognitive load	✓	~	~	✓✓	✗
Monthly cost at current usage	~$95–131	~$70	~$40–74	~$30–65	~$145+
NZ SME procurement familiarity	✓✓ AWS	✓✓ AWS	✓✓ MS	✓	✓
Future lock-in risk	~	~	~	~	✓✓
Time to first deploy from today	~5 days	~10 days	~7 days	~5 days	~8–12 weeks

5. Final Recommendation — Top Two

Given the app shape (CRUD-heavy, low traffic, bursty CPU on reports, 1-dev team, data-critical ledger, NZ locality), the two strongest options are:

A AWS Refined GOLD — FASTEST PATH TO PRODUCTION-SAFE

Best for: the fastest path to a safe, correct production system with minimal risk.

- Keeps 100% of the existing domain code intact.
- Fixes every production gap (HTTPS, secrets, real crons, private DB, migrations, CI/CD, backups, alarms) in a single focused ~5-day effort.
- The existing SST/Pulumi IaC stays; only deltas are added.
- Ongoing cost is **~$95/mo single-AZ** or **~$131/mo multi-AZ**, predictable and flat.
- No cold starts. Report pages stay snappy.
- Team retention: developer continues using the AWS tools they already know.

> **Pick over B if:** you want to ship a hardened production system in a week, you don't want to learn a new cloud, you're prepared to pay for always-on compute.

B GCP Cloud Run SILVER — LOWEST TCO

Best for: long-term lowest TCO with the simplest operator model.

- Scale-to-zero: idle overnight costs pennies.
- Fastest cold starts of any scale-to-zero option (200–800 ms typical).
- Managed HTTPS and custom-domain certs are one-liners.
- Cloud Scheduler is the cleanest cron primitive of the five options.
- The operator surface is smaller than AWS — fewer services, simpler IAM.
- Ongoing cost at low usage: **~$30–65/mo** depending on whether the Global HTTPS LB is used.

> **Pick over A if:** you want the lowest possible monthly bill, you're comfortable learning GCP, you value operational simplicity over platform familiarity, procurement has no AWS mandate.

Why not the others

- **Option 2 (AWS Serverless):** Tempting for scale-to-zero, but Aurora Serverless v2's $43/mo floor eliminates the cost advantage at this scale, and OpenNext adds a debugging layer the team doesn't need. If traffic grows 10× this becomes the best option.
- **Option 3 (Azure Container Apps):** Technically excellent, competitive on price. Recommend over GCP only if there's a Microsoft 365 / Entra ID procurement reason.
- **Option 5 (Split):** The right answer if this app were 10× bigger, multi-team, or multi-product. Wrong answer for a 1-dev team shipping an inventory ERP. Revisit in 18 months if the business warrants it.

6. Rebuild Prompt — Option A (AWS Refined)

Hand this brief to a development team — fresh or continuing — to rebuild the app on AWS with every gap closed.

Rebuild Brief: NZ Inventory ERP on AWS (Fargate + RDS + HTTPS + EventBridge)

Product: Multi-tenant B2B distributor ERP for a New Zealand health-products company. Users manage products, stock, batches, suppliers, customers, purchase orders, sales orders, proforma invoices, credit notes, reservations, 15 reports, and a QuickBooks Online sync. Current users are in the low tens; architecture must support 10×.

Goal: Production system on AWS `ap-southeast-2`, HTTPS end-to-end, private database, managed secrets, real scheduled jobs, safe migrations, CI/CD, backups, and alarms. Preserve the existing codebase; change only what must change.

Non-goals: Rewriting Server Actions, changing Prisma, introducing microservices, adding a queue, replacing the frontend.

Source of truth: Existing Next.js 15 App Router + Prisma 5 + PostgreSQL repo. Retain every page, every server action, every Prisma model, every report, every API route, all iron-session auth logic, all fiscal-year / NZT behaviour, and the QuickBooks Online integration with AES-256-GCM-encrypted refresh tokens.

Target Architecture

- **Region:** `ap-southeast-2`.
- **Edge:** Route 53 hosted zone → CloudFront distribution → ALB on HTTPS only (:443). ALB :80 redirects to :443. ACM cert auto-renewed. HSTS header set by the app.
- **Compute:** ECS Fargate, arm64, `0.5 vCPU / 1 GB`, desired 1, min 1, max 3. Health check `/api/health`. Task execution role reads from Secrets Manager. Task role has least-privilege S3 + Secrets Manager access only.
- **Database:** RDS PostgreSQL 16, `db.t4g.small` multi-AZ, 50 GB gp3, automated backups 7 days, PITR on, private subnets, `publicly-accessible=false`, deletion protection on, Performance Insights on.
- **Networking:** VPC with 2 public subnets (ALB, NAT) and 2 private subnets (Fargate, RDS) across 2 AZs. One NAT Gateway. SGs: ALB 80/443 public; Fargate 3000 from ALB SG only; RDS 5432 from Fargate SG only.
- **Secrets:** Secrets Manager — one secret per value: `session-secret`, `qbo-client-id`, `qbo-client-secret`, `qbo-encryption-key`, `database-url`, `resend-api-key`, `cron-secret`. Wired via task-def `secrets` block.
- **Storage:** S3 bucket `nz-inv-artifacts-<account>-<region>`. SSE-S3, public access blocked, versioned. Folders `reports/{tenantId}/{date}/`, `product-images/{tenantId}/`, `invoices/{tenantId}/`. CloudFront OAI for signed-URL delivery.
- **Cron:** 4× EventBridge Scheduler — each `cron(... Australia/Auckland)`:
  - `qbo-sync` 08:00 NZT → ECS RunTask with `nz-inv-cron` task def, env `CRON_HANDLER=qbo-sync`.

- `low-stock` 19:00 NZT.
- `fx-rates` 18:00 NZT.
- `reports` 07:00 NZT.

Cron task def mirrors web task def but overrides entrypoint to `node scripts/cron-handler.js`.

- **Registry:** ECR immutable tags `git-<sha>`. Lifecycle keeps last 20.
- **CI/CD:** GitHub Actions on `main` — build arm64 image, ECR push with commit SHA, `aws ecs update-service --force-new-deployment`, wait for steady state, smoke-test `/api/health`.
- **Observability:** CloudWatch Logs retention 30 d on Fargate + cron log groups. Alarms on ALB 5xx >1%, Fargate CPU >80%, Memory >80%, RDS storage <10% free, RDS CPU >80%, cron failures. Minimal dashboard. Container Insights optional.
- **Migrations:** Drop `prisma db push --accept-data-loss`. Commit migrations to `prisma/migrations/`. CI runs `prisma migrate deploy` as one-shot ECS task before rolling the web service.
- **IaC:** Keep SST 4.7 + `@pulumi/aws` in TypeScript. One `sst.config.ts` expressing every resource above. `sst deploy --stage prod` idempotent.

Code changes required (minimum set)

1. `lib/session.ts` — remove dev fallback path in prod; `SECURE_COOKIE` keyed on `APP_URL` starts with `https://`.
2. `lib/storage.ts` (new) — S3 client; `putReport`, `getReportSignedUrl`, `putProductImage`. Replace all `lib/supabase/admin.ts` usage.
3. `lib/secrets.ts` (new, optional) — at boot, read secrets from env (injected by ECS).
4. `scripts/cron-handler.ts` (new) — `switch (process.env.CRON_HANDLER) { ... }` dispatching to the four existing route handlers.
5. `app/api/cron/*` — keep for manual invocation; make handlers importable so cron-handler can call them directly.
6. `vercel.json` — delete.
7. `middleware.ts` — no change.
8. Dockerfile — drop `prisma db push` from startup CMD; startup is `node server.js`.
9. `package.json` — add `"migrate:deploy": "prisma migrate deploy"`.
10. `.github/workflows/deploy.yml` (new) — checkout, `docker buildx` arm64, ECR push, `prisma migrate deploy` one-shot ECS task, `aws ecs update-service`, smoke test.

Acceptance criteria

- `curl -I https://<domain>/` returns HTTP/2 307 redirect to `/login`.
- `curl -I http://<domain>/` returns 301 to `https://<domain>/`.
- SSL Labs scores A or better.
- Direct Postgres from public internet times out.
- Task definition JSON shows no plain secret values; Secrets Manager returns them.
- All 4 EventBridge schedules visible; manual fire writes ExchangeRate / sends alert / drains QboSyncJob / writes ReportDelivery.
- `prisma migrate status` clean; no `--accept-data-loss` anywhere.
- Bad deploy rollback via `aws ecs update-service --task-definition <previous>` in <2 min.
- CloudWatch dashboard live; synthetic alarm fires to SNS.

Deliverables

- `sst.config.ts` with full resource graph.
- Cleaned Dockerfile.
- `lib/storage.ts`, `scripts/cron-handler.ts`, `lib/secrets.ts`.
- GitHub Actions workflow.
- Data migration from current RDS to new RDS (pg_dump / pg_restore, documented runbook).
- Runbooks: `docs/runbooks/deploy.md`, `rotate-secrets.md`, `backup-restore.md`, `incident-response.md`.
- One-page architecture diagram & cost sheet.

Constraints

- Preserve all 15 reports and their exports — no regression in PDF/XLSX output.
- Preserve iron-session cookie auth. Do not introduce Cognito.
- Preserve multi-tenant isolation through app code; no RLS in this pass.
- Preserve NZT fiscal-year logic (April–March) and all date formatting.
- Preserve QBO OAuth flow and AES-256-GCM refresh-token encryption.
- Don't touch Prisma models unless adding a migration-safe constraint.
- No Vercel.

Estimated effort

Experienced AWS engineer familiar with Next.js + SST: **5–7 working days**.

7. Rebuild Prompt — Option B (GCP Cloud Run)

Hand this brief to a development team to rebuild the app on GCP.

Rebuild Brief: NZ Inventory ERP on GCP (Cloud Run + Cloud SQL + Cloud Scheduler)

Product: Multi-tenant B2B distributor ERP for a New Zealand health-products company. Users manage products, stock, batches, suppliers, customers, purchase orders, sales orders, proforma invoices, credit notes, reservations, 15 reports, and a QuickBooks Online sync. Current users are in the low tens; architecture must support 10×.

Goal: Production system on GCP `australia-southeast1` (Sydney), scale-to-zero Cloud Run + private Cloud SQL + Cloud Scheduler crons + Secret Manager + Cloud Storage. Lowest sensible TCO with managed HTTPS and a clean operator model.

Non-goals: Rewriting Server Actions, changing Prisma, introducing microservices, adding a queue, replacing the frontend.

Source of truth: Existing Next.js 15 App Router + Prisma 5 + PostgreSQL repo. Retain every page, every server action, every Prisma model, every report, every API route, all iron-session auth logic, all fiscal-year / NZT behaviour, and the QuickBooks Online integration with AES-256-GCM-encrypted refresh tokens.

Target architecture

- **Region:** `australia-southeast1` (Sydney). All services colocated to minimise egress and latency to NZ (~30 ms).
- **Project:** one per environment — `nz-inv-dev`, `nz-inv-prod`.
- **Edge:** Custom domain → external HTTPS Load Balancer with Google-managed SSL cert → serverless NEG pointing to Cloud Run service. Cloud Armor policy with baseline OWASP rules.
- **Compute:** Cloud Run service `nz-inv-web`, 2nd gen, container image from Artifact Registry, 1 vCPU / 1 GB, concurrency 20, min-instances 0, max-instances 5, CPU always-allocated off, startup CPU boost on. Ingress "Internal and Cloud Load Balancing".
- **Database:** Cloud SQL for PostgreSQL 16, `db-g1-small` (1 vCPU, 1.7 GB), 20 GB SSD, HA on, automated backups 7 d, PITR on, private IP only, maintenance window outside business hours.
- **Networking:** Default VPC with a `/28` Serverless VPC Connector in the same region. Cloud Run attaches to the connector for egress. Cloud SQL on private IP. Optional Cloud NAT for outbound internet (Resend, Frankfurter, QBO).
- **Secrets:** Google Secret Manager — one per value. Wired into Cloud Run via `--set-secrets=NAME=SECRET_NAME:latest`.
- **Storage:** Cloud Storage bucket `nz-inv-artifacts-<project>`. Uniform bucket-level access, versioning on, location `australia-southeast1`. Signed V4 URLs for delivery.
- **Cron:** 4× Cloud Scheduler jobs, timezone `Pacific/Auckland`:
  - `0 8 * * *` — POST `/api/cron/qbo-sync` with OIDC token; route verifies email matches `cron-invoker` service account.
  - `0 19 * * *` → `/api/cron/low-stock`.
  - `0 18 * * *` → `/api/cron/fx-rates`.

- `0 7 * * *` → `/api/cron/reports`.

- **Registry:** Artifact Registry `nz-inv/web`, immutable tags `git-<sha>`.
- **CI/CD:** GitHub Actions on `main` — build, push to Artifact Registry, run Cloud Run Job `nz-inv-migrate` (`prisma migrate deploy`), `gcloud run deploy --no-traffic`, verify, shift 100% traffic.
- **Observability:** Cloud Logging 30 d retention. Dashboards for req count, p50/p95/p99 latency, error rate, DB CPU, connections, VPC connector throughput. Alerts on error >1%, p95 >2 s, DB storage <10%, DB CPU >80%, failed revisions, Scheduler failures. Uptime check on `/api/health`.
- **Migrations:** Drop `prisma db push --accept-data-loss`. Commit migrations. CI runs `prisma migrate deploy` via `gcloud run jobs execute` before deploying the new revision.
- **IaC:** Replace SST/Pulumi AWS with Terraform (preferred) or Pulumi `@pulumi/gcp`. Every resource expressed as code; `terraform apply` idempotent.

Code changes required (minimum set)

1. `lib/session.ts` — `SECURE_COOKIE` keyed on `APP_URL` starts with `https://`; remove AWS-specific comments.
2. `lib/storage.ts` (new) — GCS client via `@google-cloud/storage`; put/get helpers; replace all `lib/supabase/admin.ts`.
3. `app/api/cron/*` — verify incoming OIDC token (Bearer), confirm email claim matches `cron-invoker` SA; fall back to `CRON_SECRET` header for dev.
4. `vercel.json` — delete.
5. `middleware.ts` — no change.
6. Dockerfile — drop `prisma db push` from CMD.
7. `package.json` — add `"migrate:deploy": "prisma migrate deploy"`.
8. `.github/workflows/deploy.yml` (new) — checkout, auth to GCP via workload identity federation, build, push to Artifact Registry, `gcloud run jobs execute nz-inv-migrate --wait`, `gcloud run deploy --no-traffic`, smoke test, shift traffic.
9. `terraform/` (new) — VPC connector, Cloud SQL, Secret Manager, Cloud Storage, Artifact Registry, Cloud Run service & Job, 4× Cloud Scheduler jobs, LB + managed cert, Cloud Armor, monitoring alerts, uptime check.

Acceptance criteria

- `curl -I https://<domain>/` returns 307 redirect to `/login` over TLS 1.2+.
- Cloud SQL has no public IP; connection from outside the VPC fails.
- `gcloud secrets list` shows 7 secrets; service describe does not reveal values.
- All 4 Cloud Scheduler jobs visible with NZT triggers; manual run writes expected side-effects.
- `prisma migrate status` clean; no `--accept-data-loss`.
- Rollback via `gcloud run services update-traffic` in <30 s.
- Uptime check passes; alert fires on deliberate `/api/health` failure and auto-resolves.
- Cold-start <1 s static, <2 s DB-backed after 30-min idle.

Deliverables

- Terraform modules under `terraform/`.
- Cleaned Dockerfile.
- `lib/storage.ts`, updated cron handlers with OIDC verification.

- GitHub Actions workflow using workload identity federation (no long-lived keys).
- Data migration AWS RDS → Cloud SQL (pg_dump over jump-box, documented runbook).
- Runbooks: `docs/runbooks/*`.
- Architecture diagram & cost sheet.

Constraints

- Preserve all 15 reports — no regression in PDF / XLSX.
- Preserve iron-session cookie auth. No Identity Platform in this pass.
- Preserve multi-tenant isolation through app code.
- Preserve NZT fiscal-year logic.
- Preserve QBO OAuth flow & AES-256-GCM.
- Don't touch Prisma models.
- No Vercel.

Estimated effort

Experienced engineer, comfortable picking up GCP: **5–7 working days**. Add 2 days for a team new to GCP.

8. Appendix — Honest Caveats

- **No hallucinated features.** Every capability listed in Section 1 maps to real files in the repo.
- **Pricing is indicative, not contractual.** On-demand list prices for April 2026 in the stated regions. Enterprise discounts, savings plans, committed-use, and free tiers move these ±10–30%.
- **Cold-start figures are industry medians** for Next.js containers of this size — not benchmarked on this app.
- **Aurora Serverless v2 floor is real** — cannot pause below 0.5 ACU as of writing.
- **GCP region caveat.** `australia-southeast1` is Sydney. NZ latency via the Tasman ~30 ms. No NZ hyperscaler region exists today.
- **Split option (5) is real and sometimes right** — just not for this app at this stage. Revisit if the business bifurcates.
- **Migration risk.** Whichever target is chosen, data migration from current RDS is the single riskiest step. Plan a rehearsal on a restored snapshot, a cutover window, read-only during cutover, and a documented rollback.

9. Appendix — Short-Term Fixes If Not Rebuilding Yet

If the rebuild can't happen this sprint, these three changes close the highest-severity gaps in the current stack in ~1 week:

1. **HTTPS on the ALB** (~2 days): Route 53 zone + ACM cert + ALB :443 listener + :80 → :443 redirect. Update `APP_URL` in task def so the session cookie flips to `Secure`.
2. **Move secrets to Secrets Manager** (~1 day): 7 secrets, update task def's `secrets` block, remove from plain env.
3. **Real crons via EventBridge Scheduler** (~2 days): 4 rules, `nz-inv-cron` task def with `CRON_HANDLER` dispatch, `scripts/cron-handler.ts` entrypoint.

> Do these three **even if you do eventually rebuild** — they are table-stakes production hygiene.

End of report.